# CURSO DE APRENDIZAGEM
# TÉCNICO DE INFORMÁTICA SISTEMAS Ciclo de Formação 2023/2026

## Prova de Avaliação Final

**italk**

**PROFESSOR ORIENTADOR:**
**Marco Rodrigues**

**Nº 5469 Rafael Gato**

Lisboa, data (mês/ano da defesa da PAF)

Declaro que este relatório se encontra em condições de ser apresentado em provas públicas.

_____ O professor-orientador Lisboa, _____ de

_____ de 2026

# Agradecimentos

Gostaria de expressar a minha sincera gratidão a todos os que contribuíram para a realização deste projeto.

Em primeiro lugar, agradeço ao meu Tutor, Marco Santos, pelo acompanhamento, pela disponibilidade em transmitir os seus conhecimentos, que foram fundamentais durante o meu estágio.

Apesar de só ter conhecido a professora Cláudia Santos há menos de três meses, já a considero uma profissional excecional, bem-disposta e muito simpática.

Deixo ainda um especial agradecimento ao meu orientador e a todos os professores da ECL que me acompanharam ao longo do percurso académico, em particular a Margarida Hormigo, Rita Alves, João Matos, Diogo Moreira, Pedro Barragão, Rosário Leote e Ana Costa, pelo apoio e pela partilha generosa de saberes.

Gostaria também de destacar a amizade e o bom humor de José Cruz e Rafael Coelho, pessoas extraordinárias dentro e fora da sala de aula.

À minha família e aos meus amigos, agradeço por estarem sempre presentes e pelo incentivo constante. Em especial, destaco a amizade e o apoio de António Viana, Rafael Abreu e Leonardo Pinto.

Não posso deixar de mencionar a minha professora do 1.º ciclo, Sandra, que sempre me apoiou como uma verdadeira amiga e me motivou desde o início da minha caminhada nesta escola, dando-me forças para nunca desistir.

Finalmente, gostaria de expressar o meu mais sincero agradecimento a todos quantos, de alguma forma, contribuíram para a concretização deste projeto. Sentirei saudades de cada um e guardarei com especial apreço cada momento vivido ao longo desta jornada.

# Índice

Agradecimentos ..... 3
Glossário ..... 6
Introdução ..... 7
Objetivos do Projeto / PAF ..... 8
2.1 Objetivos Gerais ..... 8
2.2 Objetivos Específicos ..... 9
2.3 Inovação ..... 10
3. Fundamentação do Projeto ..... 11
Enquadramento e Metodologia ..... 13
Desenvolvimento ..... 13
5. Caracterização do Projeto ..... 14
Back-end ..... 15
Base de Dados: SQL ..... 15
Software Utilizado ..... 15
Linguagens Utilizadas ..... 16
Razão das Escolhas ..... 16
Pesquisa e Autoaprendizagem ..... 16
Cenários ..... 17
1 – Login / Registo ..... 17
2 – Adicionar Amigos ..... 17
3 – Sistema de Mensagens ..... 17
6. Descrição do Projeto Prova de Avaliação Final ..... 18
6.1 Apresentação do Projeto/Protótipo não funcional ..... 18
6.1.1 Página Inicial ..... 18
6.1.2 Tarefa 1/ Caso de Uso 1 ..... 19
6.1.3 Tarefa 2/ Caso de Uso 2 ..... 19
6.1.4 Tarefa 3/ Caso de Uso 3 ..... 20
6.1.5 Tarefa 4/ Caso de Uso 4 ..... 20

7. Diagrama de Entidade-Relacionamento....21
7.1 Diagrama de Fluxo de Dados....21
7.2 Diagrama de Casos de Uso....22
7.3 Diagrama de Classes....23
7.4 Base de dados....23
7.6 Código Informático Desenvolvido....27
8. Conclusão....31
9. Referências....32
Guias de aprendizagem....33
Anexos....33
Anexo I - Glossário....33

# Índice de imagens

Figura 1- mockup da página home....18
Figura 2- caso de uso 1....19
Figura 3- caso de uso 2....19
Figura 4- caso de uso 3....20
Figura 5- caso de uso 4....20
Figura 6- DER....21
Figura 7- DFD....21
Figura 8- DCU....22
Figura 9- DC....23
Figura 10- Base de dados....23
Figura 11- Tabela user_entity....24
Figura 12- Tabela friend_request....25
Figura 13- Tabela friends....25
Figura 14- Tabela messages....26
Figura 15- API....27
Figura 17- Site página home....27
Figura 18- excerto de código da home page....28
Figura 19- Página de login....28
Figura 20- Excerto de código da página do login....28
Figura 21- excerto de selector de avatar....29
Figura 22- Aba de seletor de avatar....29

Figura 23- Excerto de código da função enviar imagem..... 30
Figura 24- excerto de código da função enviar mensagem ..... 31
Figura 25- excerto de código da função mensagem lida ..... 31

# Glossário

API – Application Programming Interface; conjunto de regras que permite a comunicação entre diferentes sistemas ou aplicações.

DER – Diagrama Entidade-Relacionamento; representa graficamente as tabelas e relações de uma base de dados.

FCT – Formação em Contexto de Trabalho; estágio realizado em empresa.

IDE – Integrated Development Environment; ambiente de desenvolvimento integrado (ex: Visual Studio Code).

JSON – JavaScript Object Notation; formato leve de troca de dados.

PAF – Prova de Avaliação Final; projeto final do curso.

SQL – Structured Query Language; linguagem usada para gerir bases de dados relacionais.

POST / GET / PUT / DELETE – Ações usadas numa API: criar, ver, atualizar e apagar dados.

# Introdução

O curso de Técnico de Informática de Sistemas é muito importante hoje em dia, pois a tecnologia está presente em quase tudo o que fazemos no dia a dia. Ao longo do curso, os alunos aprendem várias áreas, como programação, bases de dados, desenvolvimento de websites e também como funcionam os sistemas informáticos. Para além disso, desenvolvem competências importantes como a resolução de problemas, o trabalho em equipa e a gestão de tempo.

No final do curso, os alunos ficam preparados para trabalhar em diferentes áreas da informática, como programação, suporte técnico ou gestão de sistemas. Este curso também serve como uma boa base para continuar a estudar ou especializar-se numa área específica.

O projeto desenvolvido chama-se italk e consiste na criação de uma aplicação de mensagens, semelhante ao estilo WhatsApp e a algumas funções do Discord. O objetivo principal é permitir que os utilizadores possam comunicar entre si de forma simples e rápida, através de mensagens em tempo real.

A aplicação também permite aos utilizadores personalizar o seu perfil, escolhendo uma foto de perfil a partir de um conjunto de imagens predefinidas. Isto torna a plataforma mais interativa e fácil de usar.

Durante o desenvolvimento do projeto, foram seguidas várias etapas. Primeiro, foram definidos os principais objetivos e escolhidas as tecnologias a utilizar, como HTML, CSS, JavaScript, Java Spring Boot e MySQL. Depois, foi criada a estrutura da aplicação e desenvolvidas as funcionalidades principais, como o sistema de registo, *login*, envio de mensagens e escolha da foto de perfil. Ao longo do processo, foram feitos testes para garantir que tudo funcionava corretamente.

Este relatório está organizado em várias partes. Primeiro é apresentado o contexto do

curso e do projeto. Depois são explicados os requisitos, as tecnologias utilizadas e como o projeto foi desenvolvido. No final, são apresentadas as conclusões, dificuldades encontradas e possíveis melhorias para o futuro.

# Objetivos do Projeto / PAF

## 2.1 Objetivos Gerais

O projeto da PAF consiste na criação de uma aplicação web chamada italk, desenvolvida para permitir a comunicação entre utilizadores de forma simples, rápida e eficiente. A ideia principal é criar uma plataforma de mensagens instantâneas, onde os utilizadores possam trocar mensagens em tempo real, de forma semelhante a aplicações já existentes.

O objetivo do italk é desenvolver uma aplicação acessível e fácil de utilizar, onde qualquer pessoa consiga comunicar com outras sem dificuldades. A plataforma destina-se a um público geral que procura uma forma prática de comunicação online. Entre as suas principais funcionalidades estão o registo e *login* de utilizadores, envio e receção de mensagens e a personalização do perfil através da escolha de uma foto de perfil predefinida.

O impacto esperado do projeto é proporcionar uma experiência de comunicação simples e eficaz, permitindo aos utilizadores interagir de forma rápida e organizada. Além disso, este projeto contribui para o desenvolvimento de competências técnicas na área da programação e do desenvolvimento web, bem como competências pessoais como organização e resolução de problemas.

Para o desenvolvimento da aplicação, serão utilizadas tecnologias estudadas ao longo do curso. O *front-end* será desenvolvido com HTML, CSS e JavaScript, garantindo uma interface simples e intuitiva. O *back-end* será desenvolvido em Java , e a base de dados MySQL será utilizada para armazenar os dados dos utilizadores.

Relativamente à metodologia de desenvolvimento, será seguida uma abordagem por

fases, semelhante ao método de cascata. Inicialmente será feita a análise de requisitos, seguida pelo planeamento. Depois será desenvolvido o código, realizados testes e, por fim, corrigidos possíveis erros. Esta organização permite garantir um desenvolvimento estruturado e eficiente.

O principal objetivo do projeto é também concluir o curso aplicando na prática todos os conhecimentos adquiridos ao longo do ano letivo e durante a Formação em Contexto de Trabalho. Pretende-se demonstrar capacidade técnica, autonomia e organização no desenvolvimento de uma aplicação completa e funcional.

## 2.2 Objetivos Específicos

- **Pesquisar informação e conteúdos necessários**

  Antes de iniciar o desenvolvimento, será realizada uma pesquisa sobre aplicações de mensagens, boas práticas de programação e tecnologias adequadas. Isto ajudará a tomar decisões mais acertadas e a criar uma aplicação funcional e fácil de utilizar.

- **Construir diagramas para planeamento e estrutura do sistema**

  Serão elaborados diagramas, como fluxogramas, casos de uso e modelo da base de dados. Estes vão ajudar a organizar o projeto, perceber melhor o funcionamento do sistema e evitar erros durante o desenvolvimento.

- **Escolher as ferramentas e tecnologias a utilizar**

  Serão utilizadas tecnologias como HTML, CSS e JavaScript para o *front-end*, com MySQL para a base de dados. A escolha destas ferramentas permite desenvolver uma aplicação funcional e adequada ao projeto.

- **Desenvolver o código da aplicação**

  Nesta fase será implementado todo o sistema, incluindo o registo e login de utilizadores, envio e receção de mensagens e a funcionalidade de escolha de foto de perfil. Esta etapa transforma o planeamento numa aplicação real.

- **Testar a aplicação**

  Serão realizados testes para verificar se todas as funcionalidades funcionam

corretamente. Também serão identificados possíveis erros ou problemas de utilização.

- **Rever o projeto**

  Após os testes, será feita uma análise ao código e ao funcionamento da aplicação para verificar se tudo cumpre os objetivos definidos e identificar possíveis melhorias.

- **Corrigir erros**

  Todos os erros encontrados durante os testes serão corrigidos, garantindo que a aplicação funcione corretamente e de forma estável.

- **Elaborar o relatório do projeto**

  Será desenvolvido um relatório onde serão descritas todas as fases do projeto, decisões tomadas, dificuldades encontradas e soluções aplicadas.

- **Guardar os ficheiros de forma segura**

  Todo o trabalho será guardado em plataformas como o GitHub, garantindo a segurança dos ficheiros e permitindo acompanhar a evolução do projeto.

## 2.3 Inovação

O principal fator de inovação do italk não está apenas nas funcionalidades, mas sim na forma como estas são organizadas e apresentadas ao utilizador. Enquanto muitas aplicações de comunicação, como o WhatsApp ou o Discord, oferecem um grande número de funcionalidades que podem tornar a experiência mais complexa, o italk aposta numa abordagem centrada na simplicidade, eficiência e usabilidade.

A inovação do projeto baseia-se na criação de uma plataforma leve e intuitiva, onde apenas as funcionalidades essenciais são disponibilizadas, reduzindo a complexidade e melhorando a experiência do utilizador. Esta abordagem permite que qualquer pessoa, mesmo com poucos conhecimentos tecnológicos, consiga utilizar a aplicação de forma rápida e sem dificuldades.

Outro aspeto inovador é o sistema de interação controlada entre utilizadores, onde a comunicação só é possível após aceitação de pedidos de amizade. Isto aumenta a

organização das conversas e contribui para uma maior segurança e controlo da rede de contactos.

Além disso, o projeto foi desenvolvido com uma arquitetura preparada para evolução, permitindo a fácil integração de novas funcionalidades no futuro, como:

- Estados como por exemplo o "a escrever", online e offline;
- Criação de grupos de conversa;
- Chamadas de vídeo e mensagens de voz;
- Sistema de encriptação de mensagens para maior segurança;
- Versão mobile da aplicação.

Desta forma, o italk diferencia-se não por reinventar o conceito de aplicações de mensagens, mas por otimizar a experiência do utilizador através de uma solução simples, organizada e escalável.

# 3. Fundamentação do Projeto

O tema do projeto consiste no desenvolvimento de uma aplicação web chamada italk, criada com o objetivo de facilitar a comunicação entre utilizadores de forma simples, rápida e organizada. A plataforma funciona como um sistema de mensagens instantâneas, permitindo a troca de mensagens em tempo real entre pessoas.

O italk foi pensado como uma mistura entre duas aplicações muito conhecidas: o WhatsApp, pela sua interface simples e fácil de usar, e o Discord, pelas suas funcionalidades mais avançadas de comunicação e organização. Assim, pretende-se criar uma aplicação com um *design* limpo e intuitivo, mas ao mesmo tempo com funcionalidades mais interessantes e completas do que uma aplicação de mensagens básica.

O site irá permitir que os utilizadores criem uma conta, façam login, personalizem o seu perfil com uma foto predefinida e conversem com outros utilizadores através de mensagens privadas. O objetivo é tornar a comunicação mais prática, rápida e acessível para todos.

O italk foi desenvolvido com a ideia de ser uma plataforma simples, mas moderna, onde os utilizadores consigam comunicar sem complicações. Pretende-se que a experiência seja fluida e agradável, facilitando a interação entre pessoas no dia a dia.

Escolhi desenvolver o italk porque considero que a comunicação online é uma das áreas mais importantes atualmente. Hoje em dia, as pessoas utilizam aplicações de mensagens constantemente, seja para fins pessoais, escolares ou profissionais. Por isso, achei interessante criar uma aplicação que combine simplicidade com funcionalidades úteis, inspirada em ferramentas que já fazem parte do nosso quotidiano.

A ideia do projeto também surgiu a partir de várias sugestões e exemplos de aplicações que fui analisando ao longo do curso e com apoio de ferramentas de pesquisa. Entre essas ideias, esta foi a que considerei mais interessante e viável de desenvolver, principalmente por permitir aplicar conhecimentos de programação web e bases de dados.

Acredito que o italk poderia ser uma aplicação útil, pois muitas vezes as pessoas precisam de uma forma rápida e organizada de comunicar, sem complicações. Esta plataforma pretende responder a essa necessidade, oferecendo uma experiência simples mas eficiente.

Além disso, este projeto permite-me aplicar de forma prática os conhecimentos adquiridos ao longo do curso, desde o planeamento até ao desenvolvimento da aplicação, incluindo a gestão de dados e a criação de interfaces.

O principal objetivo do italk é criar uma aplicação de mensagens que permita aos utilizadores comunicar de forma fácil e eficiente, promovendo uma experiência moderna, organizada e acessível.

Mais do que apenas uma aplicação de *chat*, o italk pretende oferecer uma forma prática de comunicação digital, incentivando a interação entre utilizadores e proporcionando uma experiência simples e intuitiva.

## Enquadramento e Metodologia

O projeto será desenvolvido seguindo uma metodologia semelhante ao método de cascata, em que cada fase do projeto dá origem à seguinte, garantindo uma estrutura organizada e bem definida.

As principais etapas do processo serão as seguintes:

**Análise de requisitos:** inclui a pesquisa sobre aplicações de mensagens como WhatsApp e Discord, análise das suas funcionalidades e boas práticas de desenvolvimento. Esta fase serve para perceber como estas plataformas funcionam e definir as funcionalidades que o italk irá incluir.

**Planeamento:** envolve a criação de diagramas de arquitetura, fluxogramas e casos de uso, que ajudam a visualizar a estrutura da aplicação e o funcionamento do sistema. Nesta fase também é definida a organização geral do projeto e a forma como os dados irão ser tratados.

# Desenvolvimento

Nesta fase será feita a programação do *front-end* com HTML, CSS e JavaScript, garantindo uma interface moderna, responsiva e fácil de utilizar. O objetivo é criar um design simples e intuitivo, semelhante a aplicações de mensagens já conhecidas, para facilitar a experiência do utilizador.

O *back-end* será desenvolvido com Java , sendo responsável por toda a lógica da aplicação, como autenticação de utilizadores, envio e receção de mensagens e gestão dos perfis. Será também feita a integração com uma base de dados SQL para armazenar as informações dos utilizadores e das mensagens.

## Testes e validação

Serão realizados testes às funcionalidades, ao desempenho e à usabilidade da aplicação, garantindo que tudo funciona corretamente em diferentes dispositivos e navegadores.

Estes testes servem para detetar erros e melhorar a experiência do utilizador.

### Correção de erros

Depois dos testes, o código será revisto e corrigido sempre que necessário, de forma a garantir a estabilidade, segurança e bom funcionamento da aplicação.
Durante o desenvolvimento serão aplicadas técnicas como prototipagem de interfaces, diagramas de fluxo de dados e uma boa organização da base de dados. Também será utilizado o GitHub para controlo de versões, permitindo guardar o progresso do projeto e evitar perda de ficheiros.

Enquanto autor do projeto, atuo como desenvolvedor independente. O italk é uma aplicação destinada ao utilizador final, ou seja, qualquer pessoa que queira comunicar de forma rápida e prática. Embora não seja criado para uma empresa específica, poderá futuramente ser adaptado para contextos escolares ou profissionais, funcionando como uma ferramenta de comunicação interna. O italk insere-se assim no setor da comunicação digital, com potencial de evolução e expansão para diferentes tipos de utilização.

## 5. Caracterização do Projeto

### Tecnologias Utilizadas

O desenvolvimento do site italk exigiu a utilização de várias tecnologias e ferramentas, escolhidas para garantir eficiência, simplicidade e um bom desempenho da aplicação. Estas tecnologias permitem criar uma plataforma moderna e funcional, com uma boa experiência de utilização.

O projeto foi estruturado em duas partes principais: *front-end* e *back-end*, que comunicam entre si através de uma base de dados SQL.

### Front-end

- Desenvolvido com HTML, CSS e JavaScript, garantindo uma interface responsiva e intuitiva.

### Back-end

- Responsável pela lógica da aplicação, incluindo autenticação, envio de mensagens e gestão de utilizadores.
- A comunicação entre o *front-end* e o *back-end* é feita através de pedidos assíncronos (*Fetch API*), permitindo atualizar dados sem recarregar a página.

## Base de Dados: SQL

Para armazenar os dados da aplicação foi utilizada uma base de dados SQL. Esta base de dados guarda informações como:

- Dados dos utilizadores;
- Mensagens enviadas;
- A estrutura em tabelas permite manter os dados organizados e seguros. Controlo de Versões e Armazenamento: GitHub

Durante o desenvolvimento foi utilizado o GitHub para controlo de versões e armazenamento do código.

Esta ferramenta permite:

- Guardar o progresso do projeto;
- Reverter versões anteriores;
- Evitar perda de ficheiros;
- Organizar melhor o desenvolvimento.

## Software Utilizado

- Visual Studio Code: para desenvolvimento do *front-end* (HTML, CSS e

JavaScript); ● MySQL Workbench: para criação e gestão da base de dados SQL;

● GitHub: para controlo de versões e armazenamento do projeto.

### Linguagens Utilizadas

As linguagens utilizadas no projeto foram:

- HTML – estrutura da aplicação;
- CSS – design e estilo visual;
- JavaScript – interatividade e lógica da aplicação;
- SQL – gestão da base de dados.

## Razão das Escolhas

As tecnologias foram escolhidas por serem modernas, acessíveis e muito utilizadas no desenvolvimento web atual.

- HTML, CSS e JavaScript são essenciais para qualquer aplicação web.
- JavaScript permite criar toda a lógica da aplicação de forma dinâmica.
- SQL garante uma gestão organizada e segura dos dados.
- GitHub permite manter o projeto organizado e seguro durante o desenvolvimento.

## Pesquisa e Autoaprendizagem

Durante o desenvolvimento do projeto foi necessária alguma auto aprendizagem, especialmente em:

- Integração entre *front-end* e lógica da aplicação com JavaScript;
- Comunicação com a base de dados SQL;
- Estruturação de mensagens e funcionalidades de *chat*;

- Organização de interfaces modernas e responsivas;
- Utilização do GitHub para controlo de versões.

## Cenários

### 1 – Login / Registo

O utilizador entra na página inicial do site e encontra duas opções principais:

- **Login:** caso já tenha conta, o utilizador insere o email e a password para aceder à plataforma. ● **Registo:** caso seja a primeira vez, o utilizador cria uma conta através do nome, email password.

Este processo permite garantir que cada utilizador tem um perfil próprio dentro da plataforma.

### 2 – Adicionar Amigos

Os utilizadores podem adicionar outros utilizadores através do nome e ID (nome_id).

Para isso, o utilizador pesquisa pelo nome e pelo identificador único do outro perfil e envia um pedido de amizade.

Depois de enviado o pedido, o outro utilizador recebe uma notificação de convite de amizade, onde pode escolher entre duas opções:

- **Aceitar:** o utilizador passa a ser adicionado à lista de amigos, ficando a ligação entre ambos ativa na plataforma.
- **Rejeitar:** o pedido é recusado e removido da lista de pedidos, não sendo criada qualquer ligação entre os utilizadores.

### 3 – Sistema de Mensagens

O utilizador pode enviar mensagens para outros utilizadores.

As mensagens funcionam em formato de *chat*, permitindo terem uma conversa direta.

# 6. Descrição do Projeto Prova de Avaliação Final

O projeto italk consiste no desenvolvimento de uma aplicação web destinada a facilitar a comunicação entre utilizadores através de mensagens instantâneas. A plataforma permite que cada utilizador crie uma conta, personalize o seu perfil e interaja com outros utilizadores de forma simples e rápida.

A aplicação inclui funcionalidades como registo e *login*, sistema de amigos, envio e receção de mensagens em tempo real e personalização de perfil com imagem predefinida, garantindo uma experiência intuitiva, organizada e eficiente.

O *front-end* foi desenvolvido utilizando HTML, CSS e JavaScript, proporcionando uma interface moderna e responsiva. O *back-end* é responsável pela lógica da aplicação, utiliza Java para gerir autenticação, mensagens e interação com a base de dados SQL, onde são armazenadas informações como utilizadores e mensagens.

O projeto foi desenvolvido seguindo uma metodologia semelhante ao modelo em cascata, passando pelas fases de análise, planeamento, desenvolvimento, testes e correções.

O principal objetivo deste projeto é aplicar os conhecimentos adquiridos ao longo do curso, demonstrando competências técnicas, organização, autonomia e capacidade de desenvolvimento de uma aplicação completa na área da comunicação digital.

## 6.1 Apresentação do Projeto/Protótipo não funcional

### 6.1.1 Página Inicial

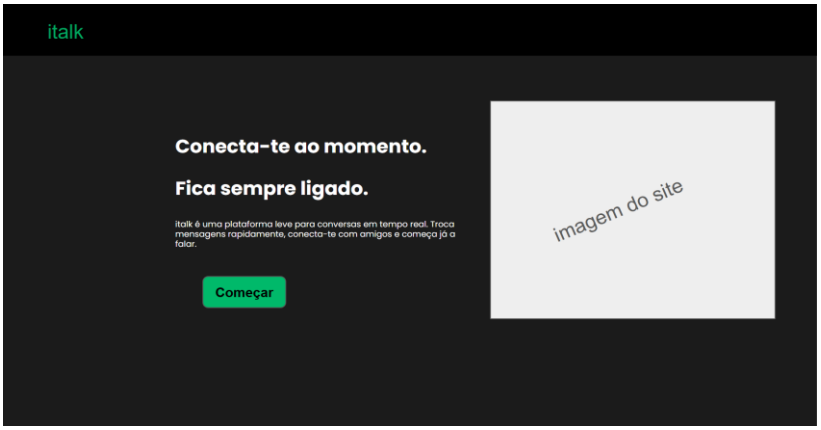


*Figura 1- mockup da página home*

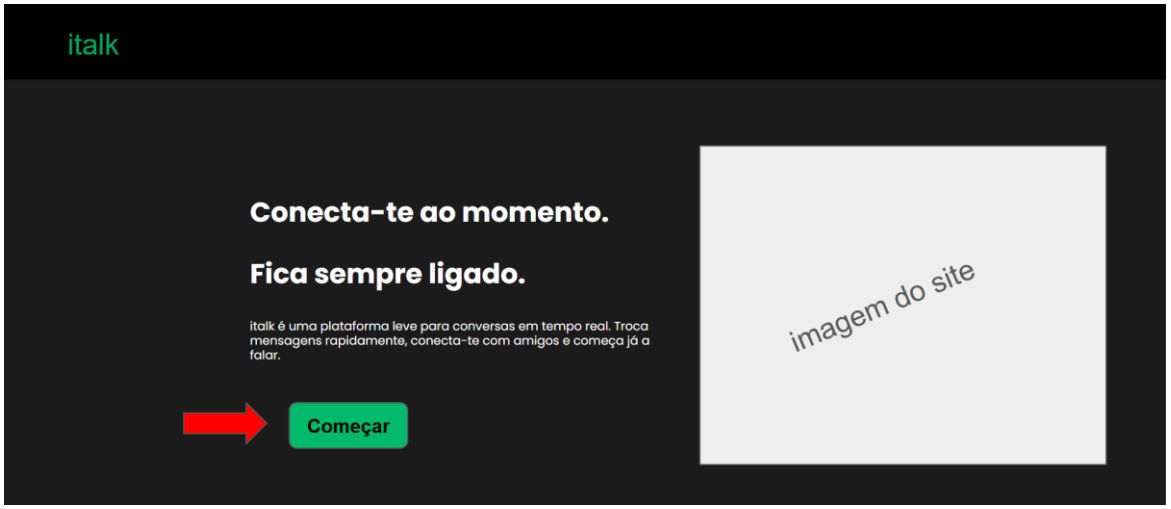


*Figura 2- caso de uso 1*

Assim que o utilizador abrir o site irá ser redirecionado para a página inicial onde terá sempre que clicar no único botão visível o "Começar".

### 6.1.3 Tarefa 2/ Caso de Uso 2

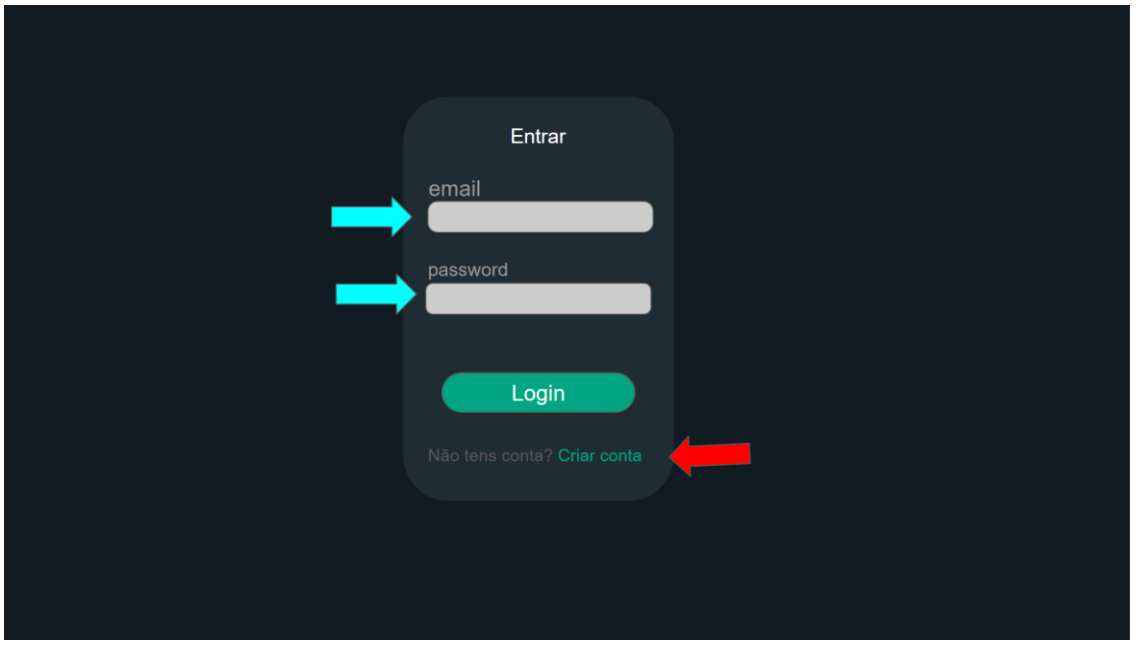


*Figura 5- caso de uso 2*

Esta é a página de *login*, onde será pedido ao utilizador o "email" e *"password"* para concluir o *login* o utilizador terá que clicar no botão verde escrito *"Login".*

Por outro lado, caso o utilizador ainda não tenha conta criada, o utilizador clica na hiperligação "criar conta" que está localizada abaixo do botão de *login.*

### 6.1.4 Tarefa 3/ Caso de Uso 3

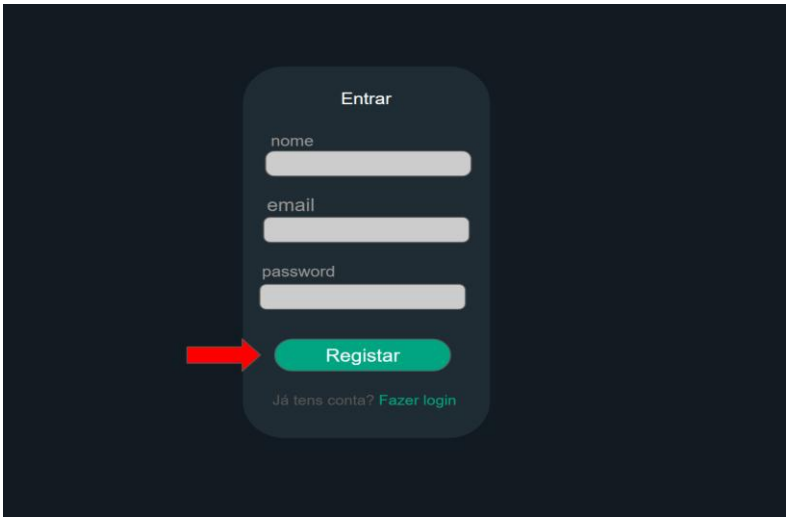


*Figura 6- caso de uso 3*

Esta é a página de registo onde o utilizador terá de introduzir o seu "nome de utilizador", "email" e por fim a *"password".* Após inserir todos esses dados o utilizador terá que clicar no botão verde escrito "Registar".

### 6.1.5 Tarefa 4/ Caso de Uso 4

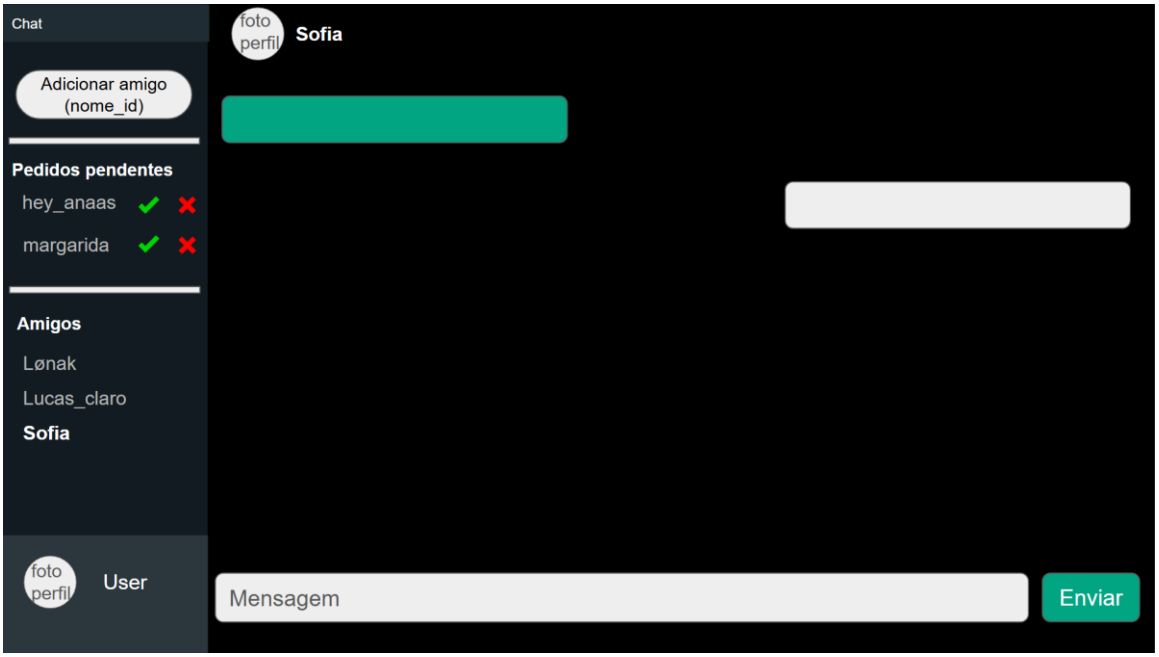


*Figura 7- caso de uso 4*

Esta será a página para a qual o utilizador é redirecionado assim que acabar o *login*, que é a página onde o utilizador tem acesso a enviar mensagens, enviar pedidos de amizade e aceitar os mesmos, ver a sua lista de amigos e eliminar conta clicando no seu nome ao lado

da sua foto de perfil.

## 7. Diagrama de Entidade-Relacionamento

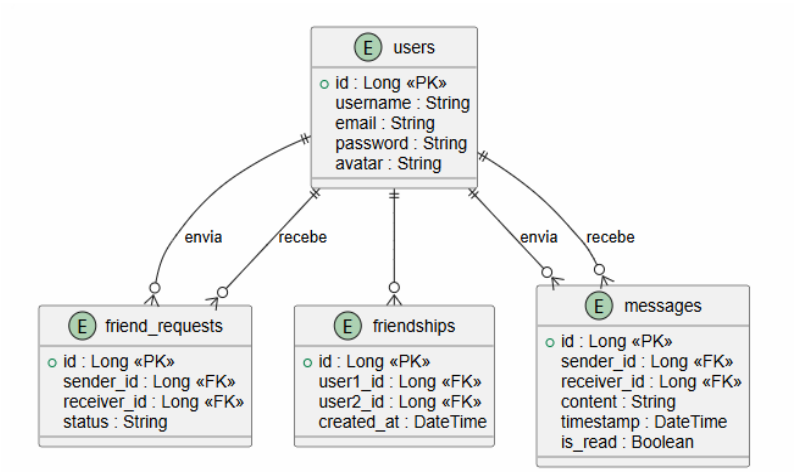


*Figura 8- DER*

Este diagrama mostra como as tabelas da base de dados se relacionam entre si. O projeto tem quatro tabelas principais: users, friend_requests, friends e messages. A tabela users guarda os dados dos utilizadores, como nome, email e password. A tabela friend_requests guarda os pedidos de amizade enviados entre utilizadores. A tabela friends regista as amizades já aceites. A tabela messages guarda todas as mensagens trocadas. As relações são simples: um utilizador pode ter muitos pedidos, muitas amizades e muitas mensagens.

## 7.1 Diagrama de Fluxo de Dados

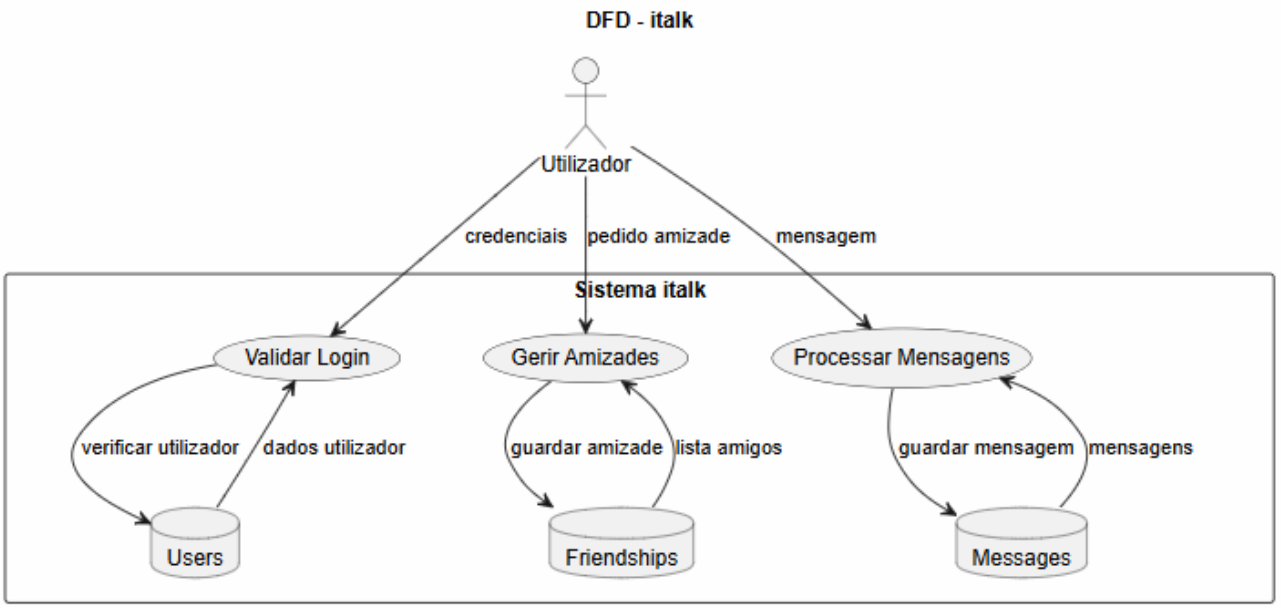


*Figura 9- DFD*

Este diagrama mostra como a informação circula dentro do sistema italk. O utilizador interage com a aplicação, os dados vão para o sistema, são processados, guardados na base de dados e depois é mostrado o resultado ao utilizador. Por exemplo, quando o utilizador faz *login,* o sistema verifica os dados na base de dados e, se estiverem corretos, entra no *chat*. Quando envia uma mensagem, o sistema guarda na base de dados e o amigo recebe.

## 7.2 Diagrama de Casos de Uso

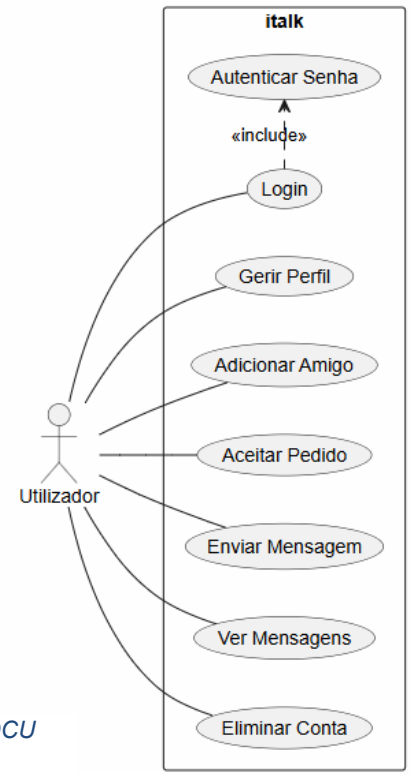


*Figura 10- DCU*

Este diagrama mostra tudo o que o utilizador pode fazer na aplicação. O utilizador pode registar-se, fazer *login*, gerir o perfil, adicionar amigos, aceitar pedidos de amizade, enviar mensagens, ver mensagens e eliminar a conta. Para eliminar a conta é preciso confirmar a password. Para aceder ao *chat* é preciso fazer *login* primeiro.

## 7.3 Diagrama de Classes

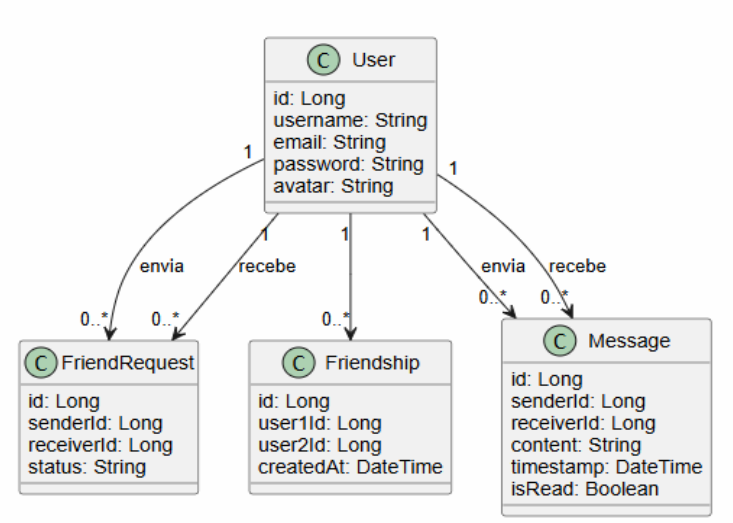


*Figura 11- DC*

Este diagrama mostra a estrutura do código do projeto. Existem quatro classes principais: User, *FriendRequest, Friendship e Message*. A classe User tem atributos como id, nome, email, password e *avatar*, e métodos como registar, *login* e atualizar. A classe *FriendRequest* guarda quem enviou e quem recebeu o pedido. A classe *Friendship* guarda os amigos de cada utilizador. A classe *Message* guarda o conteúdo da mensagem, quem enviou, quem recebeu, a data e se foi lida. As relações são simples: um utilizador pode ter muitos pedidos, muitas amizades e muitas mensagens.

## 7.4 Base de dados

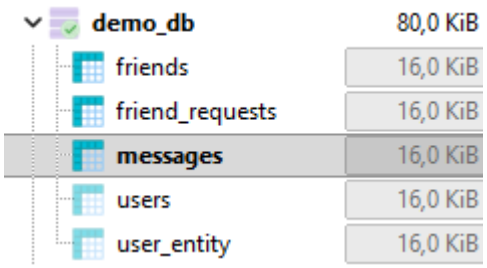


*Figura 12- Base de dados*

A aplicação utiliza uma base de dados MySQL criada no Laragon. Dentro dela existe o esquema

demo, onde está guardada a tabela users, responsável por armazenar as informações dos

utilizadores registados.

## Tabela user_entity

| id | email | password | username | avatar |
|---|---|---|---|---|
| 29 | a@gmail.com | | joaoxzao | file:///C:/Users/m3ssi/Desktop/PAF/avataresmale_black1... |
| 31 | diogo@gmail.com | 12 | diogo | file:///C:/Users/m3ssi/Desktop/PAF/avatarsmale_black.png |
| 27 | e@gmail.com | 12 | skibidi | file:///C:/Users/m3ssi/Desktop/PAF/avatarsmale_black.png |
| 33 | rafael@gmail.com | 12 | Lonak | file:///C:/Users/m3ssi/Desktop/PAF/avatarsmale_blonde... |

*Figura 15- Tabela user_entity*

Dentro da user_entity irá mostrar os dados dos utilizadores .

**id**

- É a chave primária (identificador único de cada utilizador)
- Tipo: INT (número inteiro)
- Tem AUTO_INCREMENT → ou seja, o valor aumenta automaticamente (1, 2, 3, ...)
- Não pode ser repetido

Exemplo:

- Utilizador 1 → id = 1
- Utilizador 2 → id = 2

**nome**

- Guarda o nome do utilizador
- Tipo: VARCHAR (texto)
- Tamanho máximo: 100 caracteres

Exemplo:

- "Rafael"
- "Margarida Silva"

**email**

- Guarda o email do utilizador

- Tipo: VARCHAR
- Também com limite de caracteres
- Normalmente é único (não pode haver dois iguais)

**password**

- Guarda a palavra-passe
- Tipo: VARCHAR
- Serve para autenticação (*login*)

## Tabela friend_request

| id | sender_email | receiver_email |
|---|---|---|
| 2 | a@gmail.com | qadwfc@gmail.com |

*Figura 16- Tabela friend_request*

**sender_email**

- Guarda o email de quem enviou o pedido de amizade

**receiver_email**

- Guarda o email de quem vai receber o pedido de amizade.

## Tabela friends

| id | user_email | friend_email |
|---|---|---|
| 1 | a@gmail.com | ae@gmail.com |
| 2 | ae@gmail.com | a@gmail.com |
| 3 | diogo@gmail.com | teste@gmail.com |

*Figura 17- Tabela friends*

**user_email**

- Guarda o email do utilizador que pediu ou aceitou a amizade.

**friend_email**

- Guarda o email do amigo do utilizador.

### Tabela messages

| id | sender_email | receiver_email | message | timestamp | is_read |
|---|---|---|---|---|---|
| 1 | ae@gmail.com | a@gmail.com | aaaa | 2026-04-11 17:22:50 | 1 |
| 2 | teste@gmail.com | diogo@gmail.com | wddwdwdwdwdww | 2026-04-11 17:27:42 | 1 |

*Figura 18- Tabela messages*

**sender_email**

- Guarda o email de quem enviou a mensagem.

**receiver_email**

- Guarda o email de quem vai receber a mensagem.

**message**

- Guarda o conteúdo da mensagem.

**timestamp**

- Guarda a data e hora em que a mensagem foi enviada.

**is_read**

- Guarda se a mensagem já foi lida ou não.

Tipo: BOOLEAN (0 = não lida, 1 = lida)

## 7.5 API

***Endpoints***

**/register** - Regista um novo utilizador POST

**/login** - Faz *login*

**/update-secure** - Altera nome ou password

**/delete-secure** - Elimina conta

**/friends/request** - Envia pedido de amizade

**/friends/accept** - Aceita pedido de amizade

**/friends -** Mostra lista de amigos

**/friends/requests -** Mostra pedidos recebidos

**/messages/send -** Envia mensagem

**/messages** - Mostra mensagens da conversa

**/messages/read -** Marca mensagens como lidas

**/avatar -** Atualiza foto de perfil

**/friend/remove** – Remove um amigo da listaa

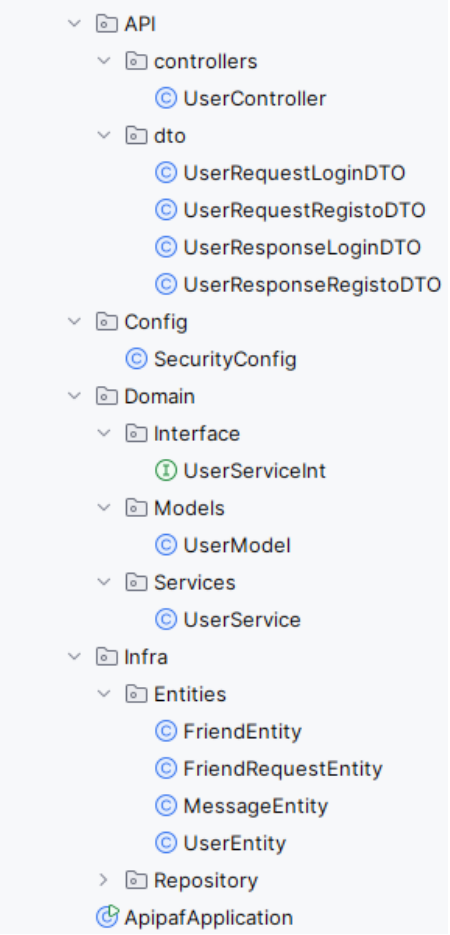


*Figura 15- API*

## 7.6 Código Informático Desenvolvido

*Figura 16- Site página home*

```
.phone-screen::after {
  content: "";
  position: absolute;
  top: 0;
  left: -50%;
  width: 50%;
  height: 100%;
  background: linear-gradient(to right, transparent, rgba(255,255,255,0.08), transparent);
  transform: skewX(-20deg);
}
```

*Figura 17- excerto de código da home page*

Este código cria um efeito visual de brilho no ecrã do telemóvel.

O ::after serve para criar uma camada extra sobre o ecrã. Essa camada é usada para representar o brilho.

O position: absolute permite posicionar esse brilho exatamente no local pretendido dentro do ecrã.

As propriedades left: -50% e width: 50% fazem com que o brilho comece fora do ecrã, do lado esquerdo, e cubra apenas uma parte, criando a sensação de movimento por cima do ecrã.

O background: linear-gradient(...) cria o efeito de luz, com uma transição de transparente para branco suave e novamente transparente, simulando um reflexo.

Por fim, o *transform: skewX*(-20deg) inclina o brilho, fazendo com que pareça mais natural, como se fosse luz a refletir no vidro.

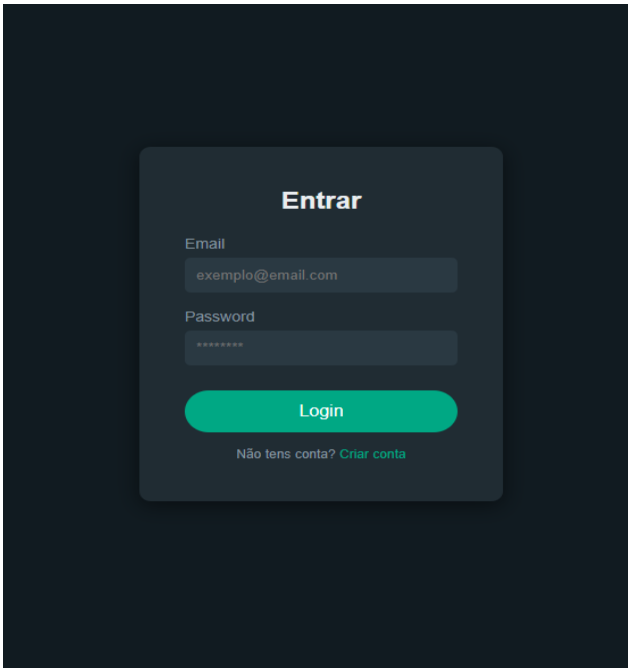


*Figura 18- Página de login*

```
fetch("http://localhost:8080/users/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    email: email,
    password: password
  })
})
.then(response => {
  if (!response.ok) {
    throw new Error("Login inválido");
  }
  return response.json();
})
.then(data => {

  console.log("RESPOSTA DO BACKEND:", data);


  localStorage.setItem("user", JSON.stringify(data));


  console.log("LOCALSTORAGE GUARDADO:", localStorage.getItem("user"));

  alert("Login feito com sucesso!");
```

*Figura 19- Excerto de código da página do login*

Este código é responsável pelo processo de *login* da aplicação. É utilizada a função *fetch* para enviar uma requisição *POST* para o servidor, contendo o email e a password do utilizador em formato *JSON*.

O servidor verifica os dados e devolve uma resposta. Caso o *login* seja válido, a resposta é convertida para um objeto JavaScript com *response.json().*

Depois disso, os dados recebidos são guardados no *localStorage,* permitindo manter a sessão do utilizador ativa no navegador.

Por fim, o código mostra uma mensagem de sucesso com *alert*("Login feito com sucesso!"), indicando que o utilizador entrou corretamente na aplicação.

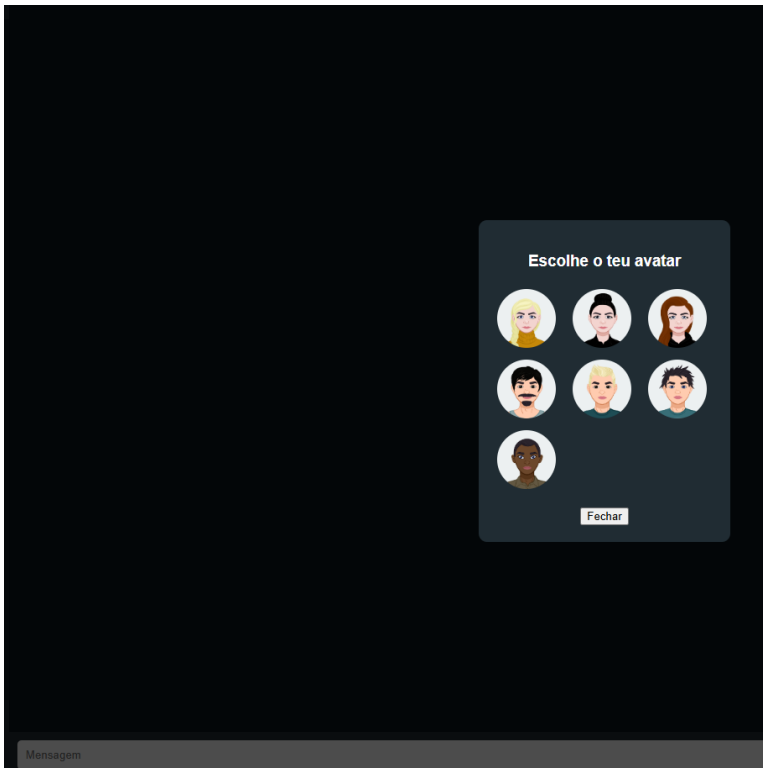


*Figura 230- Aba de seletor de avatar*

```
function selectAvatar(src) {
  const avatarDiv = document.querySelector(".avatar");

  avatarDiv.style.backgroundImage = `url(${src})`;
  avatarDiv.style.backgroundSize = "cover";
  avatarDiv.style.backgroundPosition = "center";

  const raw = localStorage.getItem("user");
  if (raw) {
    const user = JSON.parse(raw);
    user.avatar = src;
    localStorage.setItem("user", JSON.stringify(user));

    fetch("http://localhost:8080/users/avatar", {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        email: user.email,
        avatar: src
      })
    });
  }

  closeAvatarModal();
```

*Figura 22- excerto de selector de avatar*

Quando o utilizador seleciona uma imagem, a função *selectAvatar* é chamada e altera imediatamente o *avatar* mostrado na *interface* através de *backgroundImage, backgroundSize e backgroundPosition.*

Depois disso, o sistema guarda a escolha no *localStorage,* atualizando os dados do utilizador no navegador para que a imagem não se perca ao recarregar a página.

Em seguida, é enviada uma requisição *fetch* ao backend com método *PUT,* para atualizar também a imagem de perfil na base de dados, garantindo que a alteração fica guardada no sistema.

Por fim, o modal de seleção de *avatar* é fechado.

```
function formatMessage(text) {
    if (!text) return '';

    if (text.startsWith('data:image/')) {
        return `<img src="${text}" class="message-image" onclick="viewImage('${text}')" alt="Imagem">`;
    }

    let formatted = escapeHtml(text);

    const urlPattern = /(https?:\/\/[^\s]+)/g;
    formatted = formatted.replace(urlPattern, function(url) {
        if (url.match(/\.(jpg|jpeg|png|gif|webp|bmp)(\?.*)?$/i)) {
            return `<img src="${url}" class="message-image" onclick="viewImage('${url}')" alt="Imagem" onerror="this.style.display='none'">`;
        }
        return `<a href="${url}" target="_blank" rel="noopener noreferrer">${url}</a>`;
    });

    return formatted;
}
```

*Figura 22- Excerto de código da função enviar imagem*

Se a mensagem começar com data:image/, significa que é uma imagem colada pelo utilizador. A função cria uma etiqueta <img> para mostrar a imagem. Depois, procura por *links* na mensagem usando uma expressão regular (o *urlPattern*). Sempre que encontra um link, verifica se termina com extensão de imagem como .jpg ou .png. Se for imagem, mostra a imagem diretamente no *chat*. Se for um link normal, cria um link clicável. O utilizador pode clicar nas imagens para as ver em tamanho real através da função *viewImage*().

```
async function sendMessage() {
    const input = document.getElementById("messageInput");
    const message = input.value.trim();
    if (!message || !currentFriend) return;
    await sendMessageWithContent(message);
    input.value = "";
}
```

*Figura 23- excerto de código da função enviar mensagem*

Esta função pega no texto que o utilizador escreveu, verifica se não está vazio e se existe um amigo selecionado. Depois envia a mensagem e limpa a caixa de texto.

```
async function markMessagesAsRead() {
    if (!currentFriend) return;
    try {
        await fetch(`${API_BASE_URL}/users/messages/read`, {
            method: "PUT",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({
                userEmail: currentUser.email,
                friendEmail: currentFriend.email
            })
        });
```

*Figura 24- excerto de código da função mensagem lida*

Quando o utilizador abre uma conversa, esta função avisa o servidor que todas as mensagens desse amigo foram lidas. Assim o servidor pode atualizar a base de dados e remover as notificações.

# 8. Conclusão

A realização deste projeto foi uma etapa importante no percurso do curso de Técnico de Informática de Sistemas, permitindo aplicar na prática os conhecimentos adquiridos ao longo da formação. O desenvolvimento do italk representou um desafio significativo, desde a fase inicial de planeamento até à implementação final da aplicação.

Ao longo do projeto, foram alcançados os principais objetivos definidos, nomeadamente a criação de uma aplicação web funcional que permite o registo e autenticação de utilizadores, gestão de amigos, envio de mensagens em tempo real, partilha de imagens e personalização de perfil através de avatares. O sistema de amizades garante que a comunicação apenas ocorre entre utilizadores que se adicionaram mutuamente,

contribuindo para uma estrutura mais organizada e controlada.

Durante o desenvolvimento surgiram algumas dificuldades, sobretudo na integração entre o *front-end* e o *back-end*, na implementação do sistema de mensagens em tempo real e no tratamento de imagens enviadas ou coladas no *chat*. Estas dificuldades foram ultrapassadas através de pesquisa, análise de documentação e apoio de ferramentas de desenvolvimento e aprendizagem, o que permitiu aprofundar competências na resolução de problemas e na compreensão das tecnologias utilizadas.

Este projeto contribuiu para o desenvolvimento de competências técnicas importantes, como a utilização de HTML, CSS e JavaScript no *front-end*, Java Spring Boot no *back-end* e MySQL na gestão da base de dados. Para além disso, permitiu melhorar competências como organização de código, estruturação de projetos, gestão de tempo e documentação técnica.

Para o futuro, o italk poderá ser expandido com novas funcionalidades, como indicadores de utilizador online ou "a escrever", chamadas de voz e vídeo, criação de grupos de conversa e uma versão mobile da aplicação, tornando-a mais completa e adaptada a diferentes contextos de utilização.

Em conclusão, o italk corresponde aos objetivos definidos inicialmente e demonstra a aplicação prática dos conhecimentos adquiridos ao longo do curso, resultando numa aplicação funcional e estruturada na área da comunicação digital.

# 9. Referências

1.Chatgpt - Apoio para a parte da programação, tirar dúvidas sobre certas partes do relatório

2.Discord - Inspiração para algumas funções do site

3.Google/Fóruns online

4.Google Slides - Usado na projeção de como seria o site

5.MDN Web Docs - Apoio para a parte da programação mais especificamente o javascript

6.Reddit - Apoio para a parte da programação

7.Snapchat - Inspiração para a funcionalidade avatar no site

8.W3Schools - Ideias e Apoio para a parte da programação

9.WhatsApp - Funcionalidades

10.Youtube- Build A Realtime Chat App In ReactJS and NodeJS | Socket.io Tutorial (2024). Acedido em 7 de abril de 2026, em: https://youtu.be/NU-HfZY3ATQ

11.Google- Acedido em 30 de março de 2026, em: https://avatarmaker.com/

## Guias de aprendizagem

TEC3 – Desenvolvimento de API, UFCD 10791 (2025), Bruno Mateus. Curso Técnico de Informática de Sistemas. ECL – Escola de Comércio de Lisboa.

## Anexos

### Anexo I - Glossário

API – Application Programming Interface; conjunto de regras que permite a comunicação entre diferentes sistemas ou aplicações.

DER – Diagrama Entidade-Relacionamento; representa graficamente as tabelas e relações de uma base de dados.

FCT – Formação em Contexto de Trabalho; estágio realizado em empresa.

IDE – Integrated Development Environment; ambiente de desenvolvimento integrado (ex: Visual Studio Code).

JSON – JavaScript Object Notation; formato leve de troca de dados.

PAF – Prova de Avaliação Final; projeto final do curso.

SQL – Structured Query Language; linguagem usada para gerir bases de dados relacionais.

POST / GET / PUT / DELETE – Ações usadas numa API: criar, ver, atualizar e apagar dados.